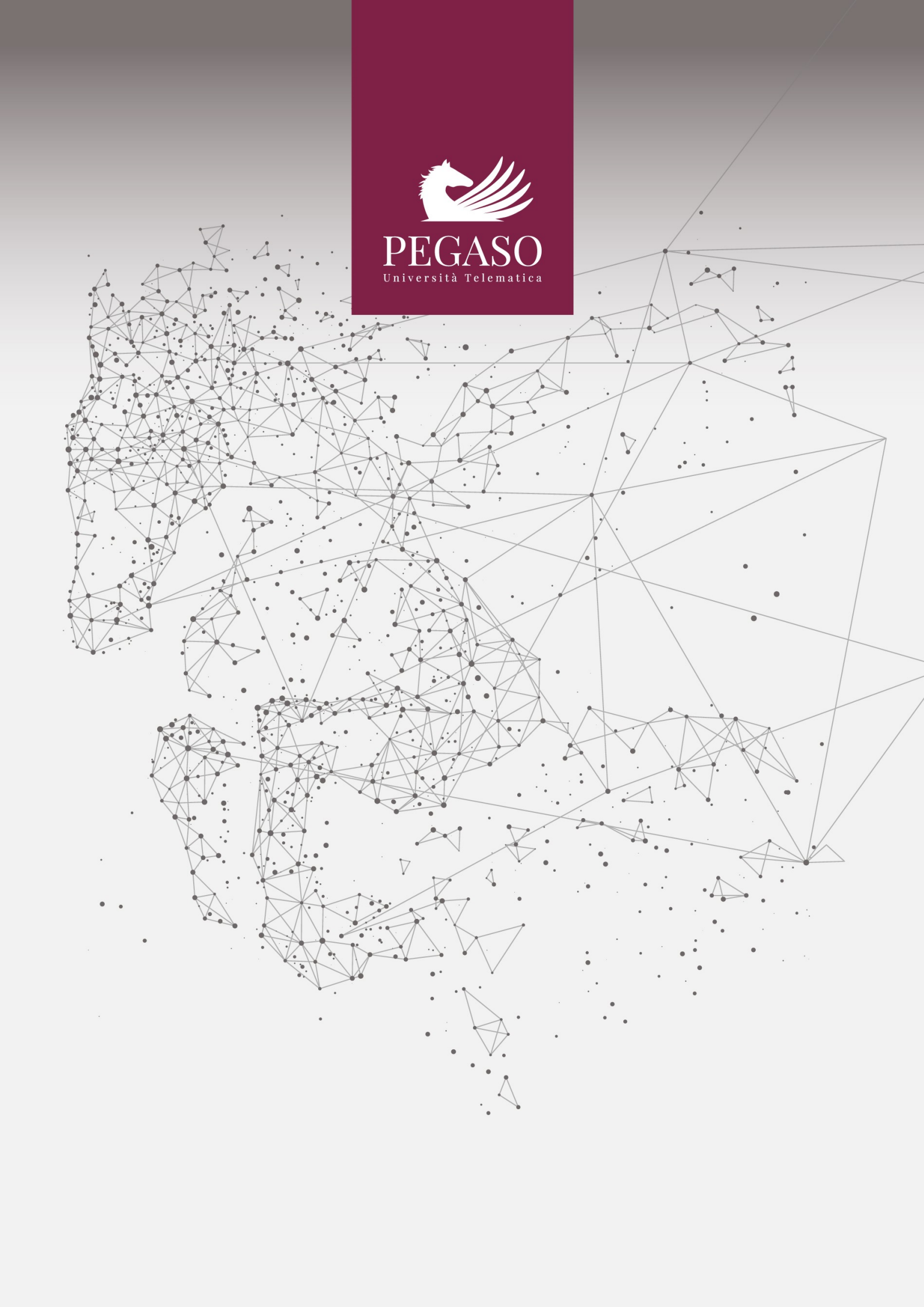

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. GRAPHICAL USER INTERFACE 4
3. INTRODUZIONE A TKINTER 5
4. SUBCLASSING DI WIDGET TKINTER 9
BIBLIOGRAFIA 18

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano il concetto di GUI e la libreria Tkinter di Python. In particolare, si illustrano le motivazioni alla base dell'uso di interfacce grafiche nell'interazione tra utente e programma e si introduce la libreria Tkinter che permette la creazione di tali interfacce per i programmi Python. Inoltre, si illustra il concetto di finestra e di widget e si presenta la struttura di un programma molto semplice di "Hello World!" con interfaccia grafica. Infine, si presenta la tecnica del subclassing dei widget Tkinter come base per la composizione di widget in interfacce più complesse.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Graphical User Interface

Nella programmazione svolta finora sono stati realizzati programmi dotati di **un'interfaccia a riga di comando** (*Command Line Interface*, ***CLI***). La ***CLI*** è un tipo di interfaccia utente caratterizzata da un'interazione testuale tra utente ed elaboratore. In particolare, l'utente impartisce comandi testuali in input mediante tastiera alfanumerica e riceve risposte testuali in output dall'elaboratore mediante display.

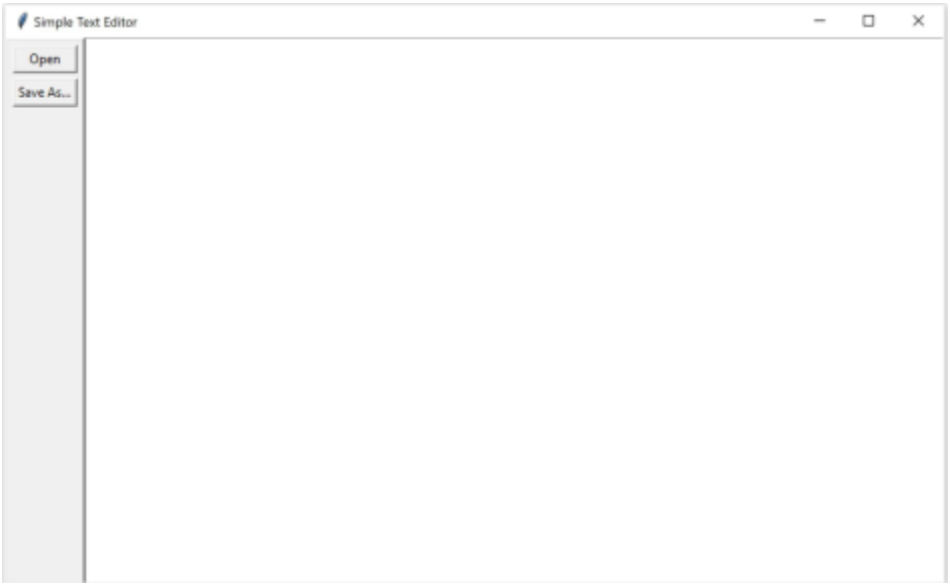
La CLI non è certo caratterizzata per essere ***user friendly*** verso gli utilizzatori dei programmi.

La caratteristica ***user friendly***, infatti, offre una misura del grado di facilità e soddisfazione con cui si compie l'interazione tra l'uomo e uno strumento.

Proprio per rendere maggiormente ***user friendly*** i programmi che si scrivono con Python, si presenta una libreria di Python che permette di creare **interfacce grafiche** (*Graphical User Interface*, ***GUI***) per l'interazione tra utente e programma. La **GUI** è un tipo di interfaccia utente che consente l'*interazione uomo-macchina* in modo **visuale** utilizzando rappresentazioni grafiche (es. finestre e widget).

La **GUI** consente all'utente di interagire con il programma *manipolando graficamente degli oggetti* attraverso l'uso di mouse e tastiera. L'esempio più comune e importante di GUI è l'interfaccia grafica dei **sistemi operativi** "moderni" come Microsoft Windows o MacOS, che si contrappone all'interfaccia di sistemi operativi più datati come *MS-DOS* (Microsoft Disk Operating System) dotati di interfaccia a riga di comando.

Un esempio molto semplice di programma con GUI realizzabile in Python è l'editor di testi (vedi figura seguente) con le funzioni di visualizzazione di testi, apertura e salvataggio dei file di testo creati.



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Introduzione a Tkinter

Tkinter (da **Tk inter**face) è una libreria che permette di creare interfacce grafiche (*GUI*) nella programmazione con Python. Esistono diversi framework per GUI per Python, ma Tkinter è l'unico framework contenuto nella ***libreria standard*** di Python. Proprio per questo motivo viene considerato uno standard *de facto* per la GUI in Python. **Tkinter** è:

- *cross-platform*, infatti, lo stesso codice funziona su diversi sistemi operativi come ad esempio Windows, MacOS e Linux;
- caratterizzato da un primato di leggerezza e stabilità;
- parte integrante della standard library di Python e, quindi, non è necessario installare altri componenti aggiuntivi per Tkinter.

Con Tkinter gli elementi visivi vengono renderizzati utilizzando *elementi nativi del sistema operativo*, quindi, le applicazioni create con Tkinter sembrano appartenere alla piattaforma su cui vengono eseguite. La programmazione con Tkinter sarà possibile sia con l'IDLE di Python che con Jupyter Notebook.

Per approcciare a Tkinter procediamo con la scrittura di un semplice programma di **"Hello World!"** secondo le migliori tradizioni informatiche. In particolare, l'Hello World in Tkinter sarà una *finestra* (**window**) dotata dei *pulsanti* (**button**) di gestione della stessa con la scritta *"Hello World!"* come quella mostrata nella figura seguente.



Per poter utilizzare la libreria Tkinter è necessario importare:

1. la **libreria Tkinter** nel *namespace* globale attraverso la seguente istruzione che non è ottimale, ma è utile per programmi molto brevi:

   *from tkinter import **

2. la libreria dei **widget ttk** (o **themed Tk**) attraverso la seguente istruzione:

   *from tkinter.ttk import **

Il **widget** è definibile come qualsiasi ***elemento costitutivo*** di un'applicazione in una GUI e la **libreria ttk** permette sia di *aggiungere* widget molto utili sia di *migliorare l'aspetto* dei widget di base.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Successivamente all'importazione delle librerie, si dovrà istanziare l'**oggetto master** (o ***root***) dell'applicazione. Tale oggetto rappresenta la ***finestra principale*** e il ***thread di esecuzione principale*** dell'applicazione.

Per tale motivo ci sarà un'unica istanza di Tk per ciascuna applicazione. L'istruzione per istanziare l'*oggetto master* è la seguente:

root = Tk()

Una volta creato l'oggetto master, procediamo con la creazione di un nuovo **oggetto Label** (*etichetta*). Come suggerito dal nome, un oggetto *label* è un *widget* per la *visualizzazione di testo o immagini.* L'istruzione per la creazione dell'etichetta è la seguente:

label = Label(root, text = «Hello World!)

Andando ad analizzare l'istruzione precedente è possibile osservare che il primo argomento passato al **costruttore Label**() è *l'oggetto master*. I ***widget Tkinter*** sono disposti in una *struttura gerarchica* partendo dalla finestra root con ciascun widget contenuto nell'altro. Ogni volta che si crea un *widget*, infatti, il primo argomento sarà quello dell'***oggetto widget*** (o della *finestra*) che lo conterrà. Nel caso in oggetto, si inserisce l'oggetto *Label* direttamente all'interno della ***finestra principale*** dell'applicazione.

Il secondo argomento passato al costruttore *Label*() è un argomento con *keyword* ***text*** che specifica il *testo* da mostrare nell'*oggetto label* stesso.

A seguito della creazione dell'*oggetto label*, è necessario disporre tale oggetto nel *parent widget*. È possibile utilizzare il metodo **pack**() che è il più semplice dei tre **metodi di gestione della geometria** disponibili. L'istruzione da utilizzare è la seguente:

label.pack()

Infine, è necessaria un'ultima istruzione che serve a far partire il **main event loop** (letteralmente *ciclo di eventi principale).* Questo loop è responsabile per *l'elaborazione di tutti gli eventi* come la digitazione da tastiera e il click del mouse.

Il loop viene eseguito *finché il programma non è terminato* e solitamente è *l'ultima istruzione di tutti gli script Tkinter* visto che il codice scritto dopo tale istruzione non sarà mai eseguito dopo la chiusura della finestra principale. Il formato di tale istruzione è il seguente:

root.mainloop()

Il codice completo del programma *"Hello World!"* è riportato di seguito insieme alla finestra generata:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
from tkinter import *
from tkinter.ttk import *

root = Tk()
label = Label(root, text = "Hello World!")
label.pack()
root.mainloop()
```



Sempre nell'ottica di esplorare i widget principali da poter aggiungere ad una finestra, procediamo con l'inserimento anche di:

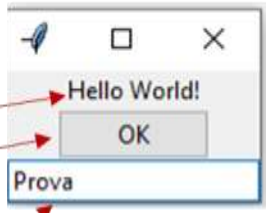
1. un **Button** che crea un pulsante cliccabile;
2. una **Entry** che crea un campo di testo da riempire.

Come per la *Label*, anche questi widget sono inizializzati con un *parent object* (anche in questo caso *l'oggetto root*) e un *parametro testuale*.

Importante è sempre richiamare il metodo **pack**() sul widget per aggiungerlo alla finestra. Il codice dopo l'aggiunta degli altri widget è:

```
from tkinter import *
from tkinter.ttk import *

root = Tk()
label = Label(root, text = "Hello World!")
label.pack()
button = Button(root, text = "OK")
button.pack()
entry = Entry(root, text = "Inserire qui il testo")
entry.pack()
root.mainloop()
```



L'aspetto dei widget della finestra risente (in positivo) della libreria dei **widget ttk** (o **themed Tk**). Provando a commentare la riga di importazione di tkk

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

*# from tkinter.ttk import **

si ottiene, infatti, il seguente aspetto dei widget e della finestra un po' più "datato".



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Subclassing di widget Tkinter

L'*approccio introduttivo* appena visto ***non è molto scalabile*** per programmi con un numero maggiore di istruzioni.

Al fine di rendere scalabile l'approccio di creazione di programmi con interfaccia grafica è necessario ricorrere al ***subclassing dei widget Tkinter*** al fine di creare *componenti widget* che verranno assemblati in una applicazione completa.

Il **subclassing**, alla base dell'***ereditarietà*** nella programmazione ad oggetti, è un modo molto semplice di:

- creare *nuove classi basate su classi esistenti*;
- creare *nuove classi modificando* solo pochi aspetti della *classe esistente.*

Per mostrare alcuni **pattern di subclassing** procediamo con la realizzazione di un programma *"Hello World"* migliorato in cui è possibile inserire un nome ed ottenere un saluto ad una persona (vedi figura seguente).



Innanzitutto, non importiamo tutte le classi, le funzioni e le variabili dalle librerie Tkinter e ttk, ma procediamo con le seguenti due importazioni:

import tkinter as tk

from tkinter import ttk

Successivamente creiamo una **nuova classe HelloWidget** come *sottoclasse* di ***Tkinter.Frame***. La ***classe Frame*** è un generico *widget Tk* tipicamente usato come **contenitore** (*container*) per *altri widget.* È possibile aggiungere qualsiasi numero di widget all'oggetto di classe Frame e trattare l'oggetto risultante come unico ***singolo widget***.

Questa modalità rende molto più semplice aggiungere pulsanti, label e entry in una singola finestra master.

Per i principi *dell'ereditarietà* della programmazione ad oggetti, una volta definita la **sottoclasse HelloWidget** di *tk.Frame* nel seguente modo:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

class HelloWidget(tk.Frame):

è necessario richiamare il *costruttore della superclasse* nella forma:

super().__init__()

La funzione ***super***(), infatti, restituisce un **riferimento alla superclasse**. Richiamando il *costruttore della superclasse* e passando gli argomenti **args* e ***kwargs*, la nuova **sottoclasse HelloWidget** può ricevere qualsiasi argomento che può essere ricevuto, a sua volta, dalla *classe Frame.*

Il codice che permette di definire la **sottoclasse HelloWidget** e il suo **costruttore** è il seguente:

```
class HelloWidget(tk.Frame):
    "Un piccolo modulo"

    def __init__(self, parent, *args,**kwargs):
        super().__init__(parent, *args, *kwargs)
```

Proseguiamo creando **due oggetti Tkinter** per memorizzare il *nome* e il *messaggio di saluto*.

Per scegliere il **tipo di dato degli oggetti** da usare è necessario fare alcune precisazioni. La libreria Tkinter è caratterizzata da una **collezione di tipi di variabili** che includono *StringVar, IntVar, DoubleVar* e *BooleanVar*. Si utilizzano questi tipi di dati al posto dei *tipi di dati built-in di Python* perché hanno la capacità di:

1. **propagare le variazioni** in modo automatico a *tutti i widget* che fanno loro riferimento;
2. **generare un evento** *quando vengono modificati.*

Il codice che permette di creare i due oggetti *name* e *hello_string* è il seguente:

```
def __init__(self, parent, *args,**kwargs):
    super().__init__(parent, *args, *kwargs)

    self.name = tk.StringVar()
    self.hello_string = tk.StringVar()
    self.hello_string.set("Hello World!")
```

Come si osserva dal codice per l'impostazione di un oggetto Tkinter è necessario il metodo ***set***() e non una semplice assegnazione (=). Allo stesso modo, per recuperare il contenuto di un oggetto Tkinter si usa il metodo ***get***().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

All'interno del costruttore, per costruire l'interfaccia grafica aggiungiamo un **oggetto Label** e un **oggetto Entry**. Il secondo parametro per il *costruttore Entry* è *textvariable.* Usando un oggetto di tipo *StringVar* (cioè *self.name*) per il parametro *textvariable*, il contenuto del **widget Entry** sarà associato all'oggetto stesso e si potrà **accedere a self.name** *senza far riferimento al widget Entry.* Il codice per l'aggiunta dell'oggetto Label e dell'oggetto Entry è il seguente:

```
        name_label = ttk.Label(self, text="Name:")
        name_entry = ttk.Entry(self, textvariable=self.name)
```

Inoltre, ogni volta che un utente scriverà del testo nell'***oggetto Entry***, la variabile *self.name* sarà ***immediatamente aggiornata*** ogni volta che appare. Il codice sviluppato finora risulta essere il seguente:

```
import tkinter as tk
from tkinter import ttk

class HelloWidget(tk.Frame):
    "Un piccolo modulo"

    def __init__(self, parent, *args,**kwargs):
        super().__init__(parent, *args, *kwargs)

        self.name = tk.StringVar()
        self.hello_string = tk.StringVar()
        self.hello_string.set = ("Hello World!")

        name_label = ttk.Label(self, text="Name:")
        name_entry = ttk.Entry(self, textvariable=self.name)
```

Procediamo, ora, creando il ***pulsante Button*** attraverso il seguente codice:

```
ch_button = ttk.Button(self, text="Change", command=self.on_change)
```

Il secondo parametro *text* rappresenta *l'etichetta che apparirà sul pulsante*, mentre il terzo parametro **command** prende il ***riferimento ad una funzione*** (o ad un metodo) Python che sarà *chiamata al click del mouse sul pulsante.*

Questo rappresenta il modo più semplice per *associare le funzioni al widget*. Si noti, infatti, che **non viene richiamata la funzione** in questo punto (*self.on_change*()), ma viene solo passato il ***riferimento alla funzione*** (*self.on_change*).

Creiamo, inoltre, un'altra ***etichetta Label*** come segue per mostrare il testo del saluto:

```
hello_label = ttk.Label(self, textvariable=self.hello_string,
    font=("TkDefaultFont", 64), wraplength=600)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

All'argomento *textvariable* di Label passiamo la *StringVar* **self.hello_string** che verrà mostrata a schermo. Questa associazione ci permette di ***modificare il testo dell'etichetta*** semplicemente *modificando* **self.hello_string**. In questo caso modifichiamo anche l'argomento font che accetta una tupla nel formato:

(nome_font, dimensione_font)

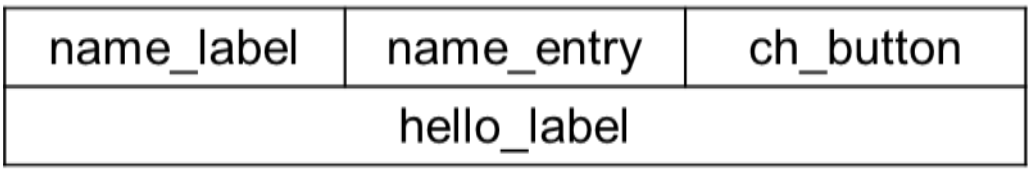
Il parametro ***wraplength***, invece, specifica quanto deve essere largo il testo prima che lo stesso passi alla linea successiva. Se tale argomento non viene impostato, il testo, di default, viene ***tagliato al limite della finestra*** (e non va a capo). Impostiamo, dunque, la *lunghezza di wrap* a 600 pixel.

Una volta creati i widget è necessario *inserirli all'interno di HelloWidget*. Procediamo dunque con l'aggiunta del widget utilizzando il **gestore della geometria *grid***() al posto del gestore utilizzato nell'esempio precedente ***pack***(). ***grid***() permette di posizionare widget nell'oggetto parent utilizzando *righe* e *colonne* come se fosse un foglio Excel.

Il codice che permette di aggiungere i widget è il seguente:

```
name_label.grid(row=0, column=0, sticky=tk.W)
name_entry.grid(row=0, column=1, sticky=(tk.W + tk.E))
ch_button.grid(row=0, column=2, sticky=tk.E)
hello_label.grid(row=1, column=0, columnspan=3)
self.columnconfigure(1, weight=1)
```

I primi tre widget (*name_label, name_entry* e *ch_button*) verranno disposti secondo tre colonne nella riga 0. Il widget *hello_label* sarà disposto, invece, nella seconda riga (riga 1). Di seguito si riporta lo schema di disposizione dei widget nella finestra.



L'argomento *sticky* può assumere valori di **direzione cardinale** (N per Nord, S per Sud, E per Est, W per Ovest) che specificano a che lato del container il **widget sarà attaccato**.

È possibile, però, attaccare un widget a più lati. Ad esempio, attaccando ***name_entry*** ad *entrambi i lati E e W*, il widget sarà *ridimensionato per riempire l'intera larghezza della colonna.*

La chiamata a ***grid***() dell'oggetto *hello_label* utilizza l'argomento *columnspan* che viene impostato a 3. Ciò implica che il widget *occuperà tre colonne della griglia*. Ciò deriva dal fatto che per la prima riga

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

vengono definite tre colonne e sarà necessario, per la seconda riga, specificare l'espansione tutte e tre le colonne se vogliamo che un widget riempia la larghezza dell'intera finestra.

Per completare il **costruttore *__init__()*** si procede ad *aggiustare la configurazione della griglia* con ***columnconfigure***(). Specificando il **peso della colonna 1** (la *seconda colonna*) ad 1, permettiamo a tale colonna di *espandersi orizzontalmente* e ridurre le colonne laterali alla loro *minima larghezza*. A tal proposito, esiste anche il metodo ***rowconfigure***() per le righe.

Il codice completo sviluppato finora è, dunque:

```
import tkinter as tk
from tkinter import ttk

class HelloWidget(tk.Frame):
    "Un piccolo modulo"

    def __init__(self, parent, *args,**kwargs):
        super().__init__(parent, *args, *kwargs)

        self.name = tk.StringVar()
        self.hello_string = tk.StringVar()
        self.hello_string.set = ("Hello World!")

        name_label = ttk.Label(self, text="Name:")
        name_entry = ttk.Entry(self, textvariable=self.name)

        ch_button = ttk.Button(self, text="Change", command=self.on_change)

        hello_label = ttk.Label(self, textvariable=self.hello_string,
            font=("TkDefaultFont", 64), wraplength=600)


        name_label.grid(row=0, column=0, sticky=tk.W)
        name_entry.grid(row=0, column=1, sticky=(tk.W + tk.E))
        ch_button.grid(row=0, column=2, sticky=tk.E)
        hello_label.grid(row=1, column=0, columnspan=3)
        self.columnconfigure(1, weight=1)
```

Prima di terminare il codice della **classe HelloWidget**, è necessario creare il metodo *callback* per il **pulsante ch_button**() denominata ***on_change***().

Tale metodo:

- prenderà il testo inserito in *name_entry* attraverso il metodo **get**() della sua *textvariable*;
- inserirà tale testo in *hello_label* al fine di mostrare il messaggio di saluto con il nome inserito.

Nel caso in cui il testo contenuto nella *name_entry* fosse **vuoto** o comprendente **solo spazi** (eliminati con metodo *strip*()), verrà mostrato il *saluto di default "Hello World".* Il codice del metodo ***on_change***() è il seguente:

```
def on_change(self):
    if self.name.get().strip():
        self.hello_string.set("Hello " + self.name.get())
    else:
        self.hello_string.set("Hello World")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si ricorda che utilizzando le *StringVar* è possibile *non accedere direttamente ai widget*. Ciò permette di evitare un *grande numero di riferimenti a widget* nella classe.

Una volta creata la **classe HelloWidget**, è necessario progettare la *classe per l'applicazione* che chiameremo **MyApplication**. Essendo una classe applicazione, essa dovrà ***ereditare*** *(*diventare cioè ***sottoclasse) da tk.Tk***.

Si precisa che per ciascuna applicazione può esserci *una sola istanza di classe Tk* e, quindi, anche *una sola istanza della classe derivata.* Anche la **classe MyApplication** chiamerà, nel costruttore, il *costruttore della superclasse Tk* passandogli gli stessi argomenti.

Questa volta, però, non sarà necessario passare un ***widget parent*** al *costruttore di Tk* perché l'oggetto Tk è la finestra *root (principale)* e non ha parent. Il codice per la definizione della funzione MyApplication è il seguente:

```
class MyApplication(tk.Tk):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
```

Dopo la chiamata al costruttore è necessario richiamare i seguenti **tre metodi per configurare la finestra dell'applicazione**:

1. **self.title**() – imposta il *titolo della finestra*;
2. **self.geometry**() – imposta la *dimensione della finestra* in pixel nel formato x*y (larghezza * altezza);
3. **self.resizable**() – imposta l'opzione in merito al *ridimensionamento della finestra* del programma.

Di seguito si riporta il codice che richiama i tre metodi. In particolare, per il self.resizable(), a titolo di esempio, viene *disabilitata la possibilità di ridimensionare la finestra* sia in larghezza sia in altezza.

```
# impostazione delle proprietà della finestra
self.title("Hello Tkinter")
self.geometry("800x600")
self.resizable(width=False, height=False)
```

La dichiarazione della classe termina *aggiungendo il widget alla finestra principale* come segue:

```
HelloWidget(self).grid(sticky=(tk.E + tk.W + tk.N + tk.S))
self.columnconfigure(0, weight=1)
```

Si noti che HelloWidget viene ***dichiarata e istanziata in una singola linea di codice***.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Solitamente si procede in questo modo quando *non è necessario mantenere un riferimento al widget*. Poiché **grid**() *non restituisce un valore*, spesso è necessario utilizzare la versione di istanziamento dell'oggetto in *due righe* in modo tale da conservare il riferimento all'oggetto e accedere al widget nel codice che segue.

Poiché l'obiettivo è quello di riempire la finestra dell'applicazione in modo completo, il metodo **grid**() fissa il ***widget*** di tipo HelloWidget a tutti i quattro lati della cella. La chiamata a ***columnfigure***() imposta la prima colonna all'espansione di tutto lo spazio della finestra.

Si noti che sono stati omessi gli argomenti *row* e *column*, quindi, **grid**() usa la *prima colonna nella prima riga disponibile* (in questo caso nella riga 0). Una volta che le classi sono state definite, è possibile scrivere il codice per la vera e propria ***esecuzione dell'applicazione***:

```
if __name__ == '__main__':
    app = MyApplication()
    app.mainloop()
```

In Python l'istruzione:

if__name__==‘__main__’:

è un'istruzione tipica per ***controllare se lo script è stato avviato direttamente***, ad esempio, da prompt utilizzando il comando:

python3 hello_world_migliorato.py

Nel nostro caso potrebbe anche essere omesso proprio perché la programmazione avviene in Jupyter. Ad ogni modo se il file del programma viene ***importato come modulo di un altro programma Python***, questo controllo restituirebbe un valore falso e il codice non sarebbe eseguito.

È buona pratica inserire il ***main del programma dopo questo controllo*** in modo tale da *riutilizzare in modo sicuro le classi le funzioni in applicazioni più estese* (cioè con più codice).

Si ricorda che ***MyApplication*** è una *sottoclasse di Tk*, quindi rappresenta la *window di base* (*root*). Sarà, infatti, necessario istanziarla e far partire il main loop. Il risultato dell'esecuzione è il seguente:



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La spiegazione appena fatta per questa applicazione abbastanza semplice è sovrastimata per l'obiettivo da raggiungere. Essa, però, è utile per dimostrare l'importanza del **subclassing** per ***segmentare l'applicazione in moduli*** che a sua volta *semplifica sia l'organizzazione sia il layout del codice* quando si scrivono programmi più complessi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice completo dell'applicazione risulta essere il seguente:

```
import tkinter as tk
from tkinter import ttk

class HelloWidget(tk.Frame):
    "Un piccolo modulo"

    def __init__(self, parent, *args,**kwargs):
        super().__init__(parent, *args, *kwargs)

        self.name = tk.StringVar()
        self.hello_string = tk.StringVar()
        self.hello_string.set = ("Hello World!")

        name_label = ttk.Label(self, text="Name:")
        name_entry = ttk.Entry(self, textvariable=self.name)

        ch_button = ttk.Button(self, text="Change", command=self.on_change)

        hello_label = ttk.Label(self, textvariable=self.hello_string,
            font=("TkDefaultFont", 64), wraplength=600)


        name_label.grid(row=0, column=0, sticky=tk.W)
        name_entry.grid(row=0, column=1, sticky=(tk.W + tk.E))
        ch_button.grid(row=0, column=2, sticky=tk.E)
        hello_label.grid(row=1, column=0, columnspan=3)
        self.columnconfigure(1, weight=1)

    def on_change(self):
        if self.name.get().strip():
            self.hello_string.set("Hello " + self.name.get())
        else:
            self.hello_string.set("Hello World")


class MyApplication(tk.Tk):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        # impostazione delle proprietà della finestra
        self.title("Hello Tkinter")
        self.geometry("800x600")
        self.resizable(width=False, height=False)

        HelloWidget(self).grid(sticky=(tk.E + tk.W + tk.N + tk.S))
        self.columnconfigure(0, weight=1)


if __name__ == '__main__':
    app = MyApplication()
    app.mainloop()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Moore A. D. (2018). Python GUI Programming with Tkinter. Packt.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).